

Distributed objects on the Web

R J Briscoe

Various distributed object technologies have traditionally been seen as necessary to protect us from the uncertainties of a world where there is a perpetual state of partial failure. The World Wide Web is the second largest distributed system in the world, behind only the telephone network which has far simpler ambitions. This paper discusses various approaches to the task of integrating the Web with more deterministic distributed object technologies to create islands of reliability (or to add other specific capabilities) without compromising the global scale of the Web. However, it is dangerous to take the view that a globally popular system such as the Web was not designed correctly. The paper goes on to explore the essence of the Web's success and discusses whether other distributed object systems would benefit from being less obsessed with deterministic behaviour.

1. Introduction

“In Chinese, the principle would be *Wei Wu Wei* — ‘Do Without Doing.’ From *Wei Wu Wei* comes *Tzu Jan* ‘Self So,’ That means that things happen by themselves, spontaneously.”

“Oh, I see,” said Pooh.

The Tao of Pooh, Benjamin Hoff

The advantages of the object-oriented (OO) approach trip off the salesman's tongue — flexibility, shorter development times, code re-use, building on the foundations of those who have gone before, reduced complexity through encapsulation, etc

Just two years ago, a learned paper on distributed systems would have been credible without any mention of the World Wide Web¹. The Web is now **the** distributed system, but it is nothing like the emerging distributed-object systems described then — those same systems that boasted that well-worn list of advantages.

So what crisis of theory produced a situation where this behemoth grew overnight defying all the design rules? Should not the efforts of the world be directed towards building the Web again properly? Or was there some hidden sting to the OO dream ticket that the Web fixes?

There are two sides to the answers to these questions, to which this paper is devoted.

- On the one hand, the deterministic distributed object systems developed over the last decade do have a large role in commercial life. However, using the ubiquitous Web as the bootstrap for such systems eases their introduction (initial human navigation or even just installation). So a large part of this paper is devoted to the issues surrounding the marriage of the two.
- On the other hand, surely it is worth seeing what this behemoth can teach us when subjected to scrutiny? Why could it not be predicted that many requirements for total reliability would be so willingly relaxed in order to achieve scaling to global proportions? Adding Web-like qualities into the formal distributed object technologies is potentially a considerably more fruitful activity than bolting the latter on to the former.

Is it enough for objects on the Web to be just the human interface bootstrapped through today's Web into relatively disconnected islands of interacting objects where there is high performance, reliability, manageability, etc?

Or is the goal a rich but still high-performance set of services provided by globally interconnected interacting machines, but where some bits do not always meet expectations? — a web of objects rather than just objects on the Web.

Or can we have the best of both worlds?

1.1 Scope

Firstly, the question: ‘What are the big issues that might prevent a web of distributed objects?’ is answered — the

¹ The author wrote one.

issues which mean that compromises are still necessary. This leads into a discussion of what an object model is, because object models are to these big distributed computing issues what political doctrines are to the big socio-economic issues.

A brief review of the most important distributed object models follows, including the Object Management Group's Object Management Architecture (OMA), Microsoft®'s Component Object Model (COM) and the Web itself. The discerning characteristics of each are highlighted. Object models are so pervasive that it is easy to live completely inside the context of just one model and be unaware of the assumptions being made. There is no best solution, rather it is 'horses for courses'. Thus developers need to understand the context in which they work, the object model that best matches this and the compromises necessary if interworking between object models is necessary.

Having covered the background, the paper starts on the core of the subject — the various ways that objects can be made to interact with the Web.

This leads into a discussion of which standardisation efforts are likely to be of most consequence to help focus the debate on the glue between objects rather than the objects themselves.

Having covered the mechanistic aspects of getting the Web and other object systems working together, the paper launches into the more philosophical analysis of the Web's strengths, the lessons that can be learned and what it would entail to re-engineer the deterministic object systems to take account of these lessons.

1.2 Audience

This paper is of primary concern to developers. The aim is to ensure that end-users and the marketing teams that nurture and satisfy their demands are insulated from decisions over object models.

Beyond the interests of developers, the lessons learned from an analysis of an organically growing global system like the Web against the criteria developed in the design of more deterministic structured systems should be of immense interest to information technology researchers.

2. The big issues

Work is the purification of the mind, not for the perception of Reality. The realisation of Truth is brought about by discrimination, and not in the least by ten millions of acts.

Shankara

The first half of this paper explains that the outstanding issues concerned with integrating the most popular object systems with the Web are approaching a solution. It is therefore inevitable that the islands of distributed objects on the Web will start to be interconnected as new capabilities provided by other islands are required. As different communities grapple with this rapid advance towards the interconnectedness of everything, it is no surprise that the scale of each previously isolated bit will appear to get greater as they each become one with the whole. This explosion of scale creates the big issues that are the subject of the second half of this paper:

- the trade-offs between performance and both flexibility and resilience as scale increases,
- the complexity that large scale brings, particularly:
 - the need for adaptation to a large variety of permutations of environment, in particular the need to find and connect together all the bits of a system (binding),
 - the associated increased risk of gridlock,
 - the development of a large system without all the developers having to understand everything,
- the management of a large system operating under federated control, which again has the same two aspects:
 - operational (run-time) management,
 - management of the build-time process which is invariably too disparate in time and space and too interwoven with other priorities to result in a total system working to commonly agreed goals, leading to the need for interoperation between differing standards approaches,
- the security of a large-scale federated system, particularly authentication of the participants and the complexity of the trust relationships.

The following sub-sections explain the nature of these big unresolved problems in more detail and may be skipped if they are already sufficiently well understood.

2.1 Performance versus flexibility and resilience

Every problem in computing science can be solved by adding an extra layer of abstraction.

Attributed to various sources

For performance, it is often preferable to fix two problems in one layer of abstraction.

Larry Masinter, Xerox Parc

At global scale, considerable flexibility is needed to isolate everyone from the continual churn that is abroad in the world. Flexibility also needs to be introduced to cater for all the permutations of everyone's differing requirements. Over the years, further and further layers of wrapping really have been added to solve any and every problem. The best designed abstractions only kick in when there is a problem or a configuration is specifically required, but during normal operation, they sit quietly to one side of the communications channels. However, the larger the scale, the more often there are little changes which need the abstraction layers to activate, and consequently real performance suffers. Worse, not every abstraction layer is well designed. Many abstractions typically kick in when a new interaction is attempted leading to latency problems, particularly when 'new' means 'new this session' with no storage of configuration across sessions.

Fault tolerance is a particular example of such an abstraction layer. Typically an interaction is with a named object, but the mapping between that name and the object's physical address can be changed if the first object fails. Under failure conditions, first the failure has to be detected which often involves a time-out, then it has to be reported so the address is no longer advertised, then a new address mapping has to be requested. Not only does this change-over all take time, but it often results in the name-to-address mapping being done on every access, if there is no address caching mechanism.

2.2 Complexity

The complexity of a large-scale system results in both run-time and build-time problems.

At run time the complexity challenge becomes one of discovery of services. The art of programming is to be non-specific and dynamic with minimal hard-coded addresses, limits or messages, while at the same time being specific enough for the code to be applied to the problem. This does not mean that the end user is expected to spend the first day configuring the system before it can be used. Best practice is for the system to sit within a well-designed framework which allows it to configure itself to the environment in which it finds itself. This is obviously easier said than done at large scale; some early stabs at this problem are described later.

Also at run time there is the specific problem of the increased risk of gridlock as system complexity and scale increases. Where objects need to make no calls to other objects in order to service requests made on them this is not a problem. However, it is common that objects rely on each other to provide a service, a situation which is prone to dependency loops.

At build time the challenge is to design a systems engineering process that allows each software engineer to create their bit without having to understand how to do

anything but the kernel of the task in hand. If in order to write a customer support system, the developer has to understand how e-mail works including all the quirks, how computer/telephony integration is done, how to secure the system against intruders, how to distribute the system across multiple machines, how to avoid race conditions corrupting the database, etc, then something is wrong with the process. Even the current solution to this problem is running into complexity problems — that is to encapsulate each of these peripheral problems with a well-known application programming interface (API). However, most APIs are reasonably complex, and each programmer has to understand a very large number of them, leading to mistakes where particular APIs are not fully understood. Solutions to this new problem are discussed later.

2.3 Management

Operational management of a very large scale system can typically be characterised as a problem of co-ordination between a large number of parties none of whom accept subordination to the control of any of the others (a federation). Where the number of parties, n , is very large such that the number of bilateral agreements between them would be proportional to n^2 , there are only two practical approaches (assuming that multiparty interactions can be distilled down to the pairs that they contain):

- pre-agree (contract) to a standard or a hierarchy of standards,
- dynamically negotiate a management contract between parties as they interact for the first time (which of course must itself be based on some standard negotiation protocol).

Any attempt to manage the design process of a large-scale system inevitably leads to failure as there are just too many interacting decisions to be made as all the requirements and priorities change over long development time-scales. This activity degenerates into one of interoperation between systems that have gone through separately managed design processes.

2.4 Security

Large-scale security involves establishing trust that the models of the interacting parties that the application presents do truly represent who they claim to and that the liability of the real parties can continue to be enforced into the future. This typically involves the creation of contractual chains of liability through networks of trusted third parties. These have been designed in theory, but the world is only just starting to build such networks. It remains to be seen what legal cracks will appear in large-scale systems and what impact on performance will transpire.

2.5 In summary

The theme of this paper is the lessons that designers of distributed object systems can learn from the scalability of the Web. As large-scale security solutions are in their infancy on the Web and the management capabilities are not particularly mature, the discussion in section 6 will focus on the performance, resilience and complexity domains.

3. Distributed object models

Can I explain the Friend to one for whom He is no Friend?

Jalal-uddin Rum i

Over time, the solutions to the big issues just introduced have coalesced into coherent conceptual approaches. The aspect of these world views that best characterises each is their 'object model'.

In general, in order to achieve some purpose or application, we create models of entities and processes. The International Standards Organisation (ISO) Open Distributed Processing Reference Model (RM-ODP)[1] (which is a framework, defining concepts and functions in open distributed processing) defines five useful viewpoints on such models, one of which is the computational viewpoint (see Fig 1). This is the perspective from which programmers create the concepts and entities they find useful to manipulate the underlying model. In the process, a computational model is created. This does not completely model the application in hand, but it is sufficient for the programmer's needs. Similarly an enterprise model would be sufficient for conceptualising the task in hand from the business manager's viewpoint (one of the other four).

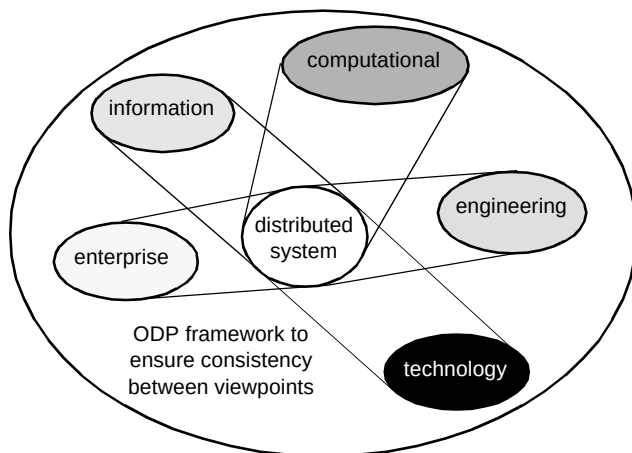


Fig 1 The Open Distributed Processing Reference Model viewpoints.

An object model is simply a computational model of an object-based system. The circular reference to the word 'object' here is the heart of the confusion between various object models because the features of an 'object', are defined within each object model. There is general agreement on a minimal definition of what an object is. It is

the representation of an application domain entity using data to represent its state associated with procedures (or methods) to represent its behaviour. However, an object model embodies wider assumptions about an object than simply what it is. For instance, all object models known to the author also agree that:

- an object encapsulates its state to prevent undisciplined access or alteration except through its published methods,
- new objects can be formed at run time (instantiation) based on a template (class) so that multiple similar objects can exist in one system each with a different state,
- each such object can be referred to by a first-class reference,
- there is infrastructure to allow a method to be invoked on many different sorts of objects without the client first having to establish which method matches which object (substitutability or inclusion polymorphism). For instance, to order a carpet, and to order a holiday could both be done with an 'order' method and the right method would be invoked based on which type of object was being ordered.

Typically, each object model includes the software analogue of the specification of a hardware bus into which circuit cards (server objects and their clients) may be plugged. It defines how a client can address a server object much like one circuit card might access another over the bus without knowing which slot it is in, be it in the same machine, or remote (location transparency).

Just as a physical bus often appears to be just wires and connectors with no active components, so it can be with object interaction specifications. Some physical bus specifications are implemented by bus interface logic on each card, others involve some central circuitry that controls the bus. So it is with 'object buses' which can be manifested as stub or skeleton code, half in the client and half in the server or alternatively as processes that arbitrate and manage the interaction of all objects.

This paper will concentrate on those aspects of more influential object models that are relevant to the big scale-related issues listed earlier. It will also assess whether these object models really do sufficiently model the application, or whether the programmer's computational model needs more from other RM-ODP viewpoints (particularly enterprise and engineering).

However, before comparing these features of the object models, each will be introduced on its own terms:

- the Object Management Group's Object Management Architecture,
- Microsoft's Component Object Model,

- Sun[®]'s Java[™].

Other significantly different object models such as those used by autonomous agent infrastructures and object databases have not been covered due to lack of space, time and expertise.

3.1 Object Management Architecture

The Object Management Group (OMG) is an international non-profit consortium with over 700 members. It aims to promote the theory and practice of distributed object technology. Its architecture [2] is generally known as the Common Object Request Broker Architecture (CORBA[™]) although strictly CORBA is merely one part of the OMA, albeit the core part.

Continuing the analogy between object buses and hardware buses started above, different firms can manufacture circuit cards that comply with a bus specification without the electronic components or circuit being standardised. Some card designs run faster, some are more reliable, etc. CORBA is similarly vendor-neutral. There is no reference implementation of CORBA, only many competing products or Object Request Brokers (ORBs) that comply to the standard.

The only job of the ORB is to intercept a call and be responsible for finding an object that can implement the request, pass it the parameters, invoke its method, and return the results. The term 'finding' here is not intended to imply some form of magic global service discovery. In keeping with CORBA's design principles [3], 'services are designed to do one thing well and are only as complicated as they need to be'. In the case of the ORB, 'finding' simply means, having got the desired object reference from some other object, accessing it and discerning the method on that object that suits the parameters. The client does not have to be aware of where the object is located, its programming language, its operating system, or any other system aspects that are not part of an object's interface. At this point the term 'interface', as used in the OMA, should be clarified. A CORBA object has just one interface, which is not the case in some other object models.

Generally, a pre-compiler is supplied with an ORB product which generates the source code of both a client stub and a server skeleton from a definition of the object interface, written in OMG Interface Definition Language (IDL). The programmer can then concentrate on writing the application source code, not the code for intercommunication. However, the CORBA specification does not mandate this approach — it would be possible for an ORB to be compliant that used separate well-known processes to manage the 'object bus'.

CORBA does not constrain the wire protocol that each vendor's ORB uses to pass messages across a network. However, compliance with the CORBA/2 [4] specification can only be achieved if there is a gateway between whatever protocol an ORB uses and the Internet Inter-ORB Protocol (IIOP).

The OMG also takes on the role of arbitrating in the standardisation of certain key object interfaces that form the complete OMA (see Fig 2). These fall into three broad categories:

- domain interfaces that are worth standardising for all software aimed at a particular vertical market (e.g. finance, manufacturing),
- Internet facilities needed so prevalently across all markets as to be worth standardising, e.g. compound document facilities, help systems,
- CORBA services are the lower-level (and thus the most crucial) object interfaces — examples are:

Naming Service,

Life Cycle Service,

Persistent Object Service,

Transaction Service

Concurrency Control Service,

Relationship Service,

Time Service,

Security Service.

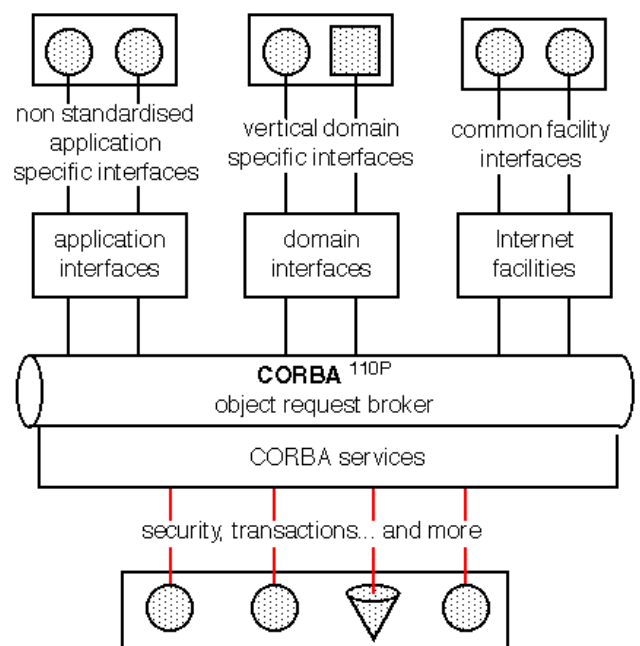


Fig 2 The Object Management Architecture.

3.2 Component Object Model

The foundation of Microsoft object services is the Component Object Model (COM) [5]. Although originally proprietary to Microsoft, the COM, as a 'core technology' of ActiveX™, has been assigned to the Open Group for standardisation. ActiveX is Microsoft's brand name for the technologies that enable interoperability using the COM.

A component in the COM is a binary entity that provides some service that may be used by application software. The COM is primarily a specification (hence model) for how objects and their clients interact through the standard of interfaces. It covers such basics as interface negotiation, reference counting, rules for memory allocation (between independently developed components) and error reporting.

A particular design goal and strength of COM is the ability to manage versions of components (specifically to permit a client to determine which versions of a service are offered by a component in 'less than one keystroke time on a PC'). COM versions are manifested as multiple interfaces of one component which have separate globally unique identifiers. Having found a component, a client can quickly establish whether it provides a compatible version of the service required.

COM enables programming language independence, but unlike in CORBA, interface interaction is specified at a binary level which reflects Microsoft's dominance of a single CPU architecture (Intel®). As well as being a specification, COM does involve an implementation as system level code (the equivalent of co-ordinating circuitry in the hardware bus analogy introduced above). Therefore it is possible for it to be ported to other architectures. Early releases of such ports to other CPU architectures are now appearing. These underlie the interoperability across the Internet that is the goal of ActiveX.

The COM began life designed for component interaction within a single computer's address space. During 1996, the distributed COM (DCOM) wire protocol was published as an Internet draft [6] and versions of COM that support it are now becoming available (currently on Windows NT® and Windows®95). Distributed security is based on the same specifications as is the Distributed Computing Environment (DCE) remote procedure call (RPC) [7] which is the basis of the DCOM RPC. COM distribution is discussed in more detail in section 6.

Being the core of object creation and management, COM also acts as a foundation for further infrastructure, such as persistent intelligent names and structured and persistent storage. On top of these are built Microsoft's object linking and embedding technology and all the other facilities specified by the ActiveX APIs.

In the other direction, COM is also the start of a path towards a system object model which Microsoft plans to be the glue binding together future versions of Windows NT. This is based on the common object model, also confusingly called COM.

3.3 Java

The term Java applies to much more than just the object-oriented programming language. It also covers the specification of the bytecode to which the source-code is compiled and the virtual machine that interprets this bytecode. Taken together, these define a complete build-time and run-time system and therefore have a distinct object model.

All these aspects of Java have been defined by JavaSoft, a business unit of Sun Microsystems Inc. However, Sun are taking preliminary steps towards standardising Java through ISO's Java Study Group (SC22). It is unclear what the term 'Java' is being used to encompass in this context.

The Java run-time system is shown in Fig 3 alongside a traditional run-time system (the object code). The bytecode specification is designed to have safety features for network download into a virtual machine (although it can be loaded from disk, of course). The virtual machine prevents any direct access to the real machine or the operating system running on it. The virtual machine hides the differences between heterogeneous operating systems (a different one is needed for each platform) so that one version of the bytecode will run on any platform. However, it is envisaged that the future will see many more examples like on the right hand side of Fig 3 where the virtual machine is tailored for particular hardware taking over the role of the operating system. This is particularly likely in less modular equipment than general-purpose computers, e.g. household appliances, communications equipment.

Bytecode is produced by 'semi-compiling' Java source code. Bytecodes are instructions for the virtual machine akin to machine instructions for real processors. Once processors are manufactured that interpret bytecode there will be no distinction.

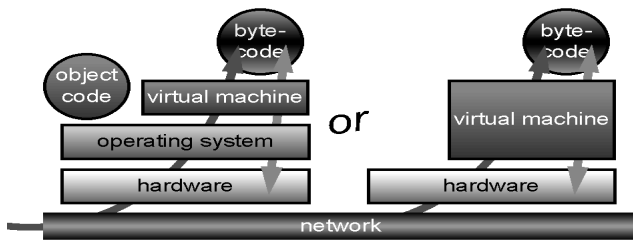


Fig 3 Traditional and Java run-time systems compared.

Java's object model involves dynamic linking. The virtual machine links Java programmes together at run time, eliminating the need for this at compile time. This is the crucial property of the Java model that makes it possible to discuss it on the same level as other object models like the OMA or the COM. However, a lot of the transparencies built on the core object models of OMA and COM are yet to be defined for Java although some of the early work in this area is discussed later. In the interim, there is no reason why Java-written applications cannot be built into the object model infrastructure provided by the OMA or the COM (and they have been). This simply adds abstractions to Java's own object model. The disadvantage is that this has a performance penalty. The advantage is language independence for each of the components making up a system.

4. Object and Web interoperation

When is a man in mere understanding? I answer, 'When a man sees one thing separated from another.' And when is a man above mere understanding? That I can tell you: 'When a man sees All in all, then a man stands beyond mere understanding.'

Eckhart

Having discussed three of the most influential distributed object models in the previous section, we now arrive at the core of the subject matter — how to put these distributed objects on the Web.

Merle [8] gives an excellent overview of the various options the system designer has for utilising the ubiquitous Web infrastructure as a doorway to more structured object systems. These approaches all get round the installation and update problems that object technology has. There is no better approach to describing the options than interspersing discussion with broadly plagiarised diagrams from this reference. To help excuse this, the figures have been generalised from being CORBA specific to include how they would be done under the COM as an example alternative, and the commentary discusses yet further object models.

The user end is characterised as a 'desktop' with a Web browser. In fact, as long as it can talk the same protocols

and has the same extensibility capabilities, it could be built into an operating system or an embedded device in some household appliance.

The first and most prevalent way (Fig 4) to integrate an object system with the Web infrastructure is through a bespoke gateway using the Common Gateway Interface (CGI) [9] that comes as standard with every webserver. Alternatively the webserver APIs can be used for performance, but these vary across suppliers. There is scope for more generalised solutions, such as a specific client that solves a whole class of problems, but otherwise this technique is not of enduring interest.

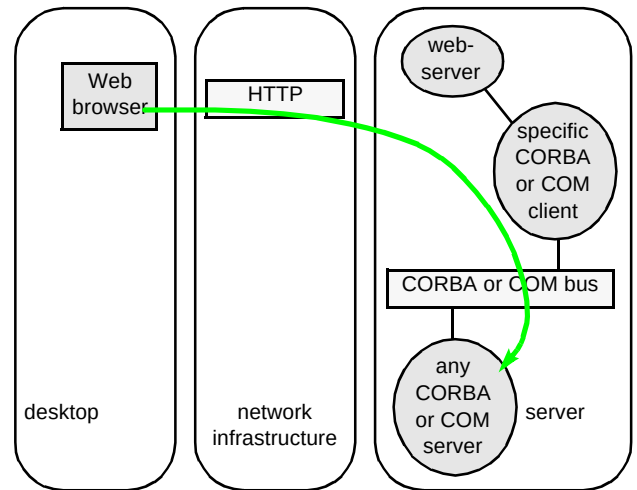


Fig 4 Integration via specific webserver gateway.

Under CORBA, it is possible to modify the above approach using the Dynamic Invocation Interface (DII) to create a completely general server-side gateway (see Fig 5). The reason this is CORBA-specific is that it relies on the existence of an interface repository where objects can register their interface types and the DII to look up the type in order to dynamically construct the call. It is possible in the COM (and possibly in other object models) to build such an interface repository function, but this requires the manual creation of an equivalent capability and modification to the development environment which otherwise discards the interface description on compilation.

The interfaces between the Web and object or object-relational databases can also be classified as generic gateways between the webserver and the object model of the database.

The approach in Fig 6 was pioneered by the ANSAweb proposal [10]. The ANSAweb code treated all resources with URLs [11] preceded by 'http://' (implying they were accessible by hypertext transfer protocol (HTTP) [12]) as potentially reachable with CORBA IIOP. The aim was to decouple the URL from the choice of transfer protocol. Ignoring the desire for this particular abstraction it is

possible to specify the object interaction protocol explicitly by extending the allowed protocol schemes for URLs. Thus the URL to access an object with CORBA IIOP would start 'IOR:' (interoperable object reference) or 'clsid:' (class identifier [13] for ActiveX). However, in practice many assumptions have become entwined in obscure areas of all browser code that make such changes away from HTTP difficult.

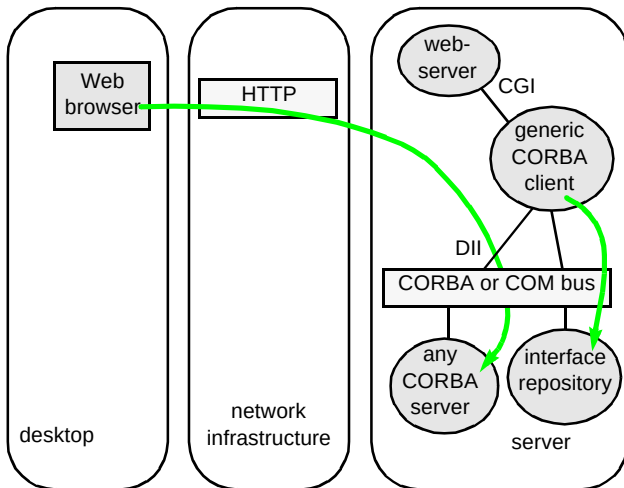


Fig 5 Integration via generic webserver gateway.

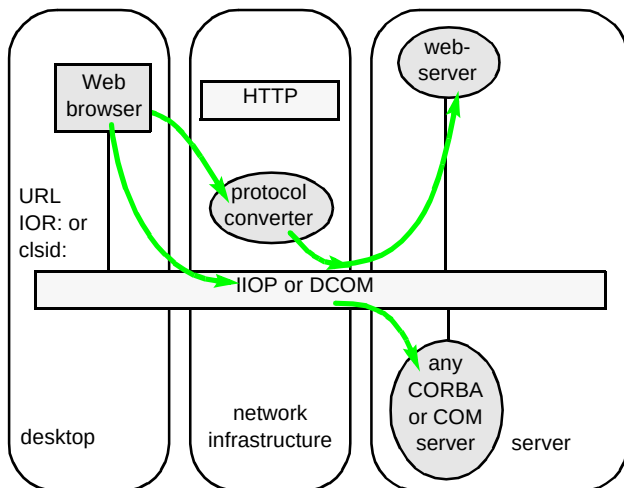


Fig 6 Integration by unifying communications protocols.

However, the primary flaw in all three schemes for integration of object systems with the Web discussed so far is that they only allow you to access object-based servers through a Web browser. Thus they are limited to the user interface devices that are built into a browser (notwithstanding that these have been put to a massive range of tasks) or specific clients that can be installed through other channels.

Figure 7 shows the simplest arrangement to overcome this problem. Here, any user interface that cannot be achieved from standard browser elements can be downloaded as mobile code (an 'applet'). This might be

Java, or a scripting language like Visual Basic Script or whatever becomes the prevalent browser standard. The important aspect is that any infrastructure elements that are not available at the user end, such as a protocol stack, can also be downloaded. It is obviously preferable that such commonly used software is not continuously downloaded and this has been recognised by the main browser vendors. This is likely to be an extremely common arrangement in the near future. Exactly which technologies are used depends on which become prevalent on the desktop. This is discussed later in section 5.

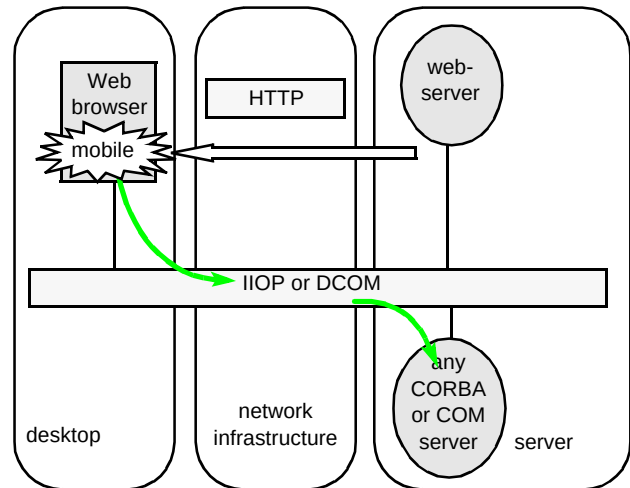


Fig 7 Integration using applets and remote objects.

Figure 8 is simply a reminder that the previous scheme is just as relevant where the server object has been installed through other channels at the user end.

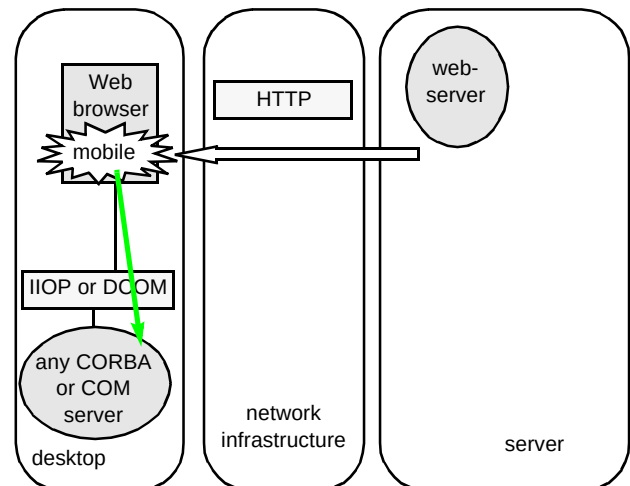


Fig 8 Integration using applets and local objects.

So far, the user end has been assumed to be causing a call. Another large class of interactions fits the retro-action or call-back model, where an interest in a certain class of reply is registered by the user end, leaving a call-back address, and asynchronously, replies or event notifications

appear any time later. Figure 9 shows how the special desktop server and its protocol stack for such interaction could be downloaded as mobile code such that notification messages could be sent to it (having previously subscribed to them). A specific server object installed by other means is shown as an alternative. Such arrangements are not yet prevalent enough for a common approach to the problem of opening up a firewall to admit asynchronous messages on connections not directly initiated from inside.

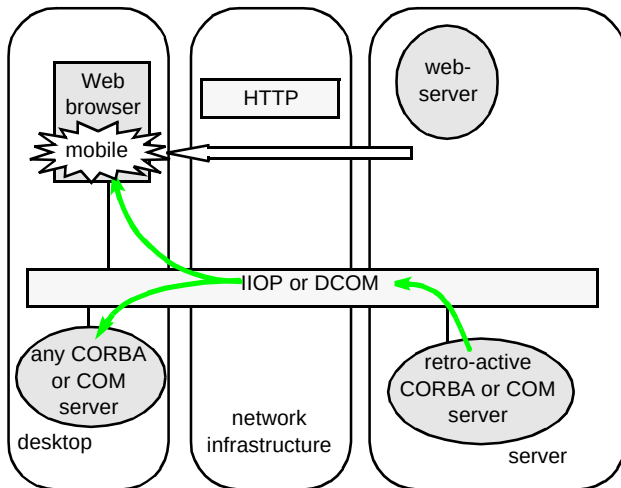


Fig 9 Integration using applets as notification objects.

Servers downloaded in this manner are obviously not confined to only being used in call-back situations; however, this is envisaged as the most common requirement.

Finally, support for a peer-to-peer interaction model is shown in Fig 10. Here, two browsers are shown using protocol elements downloaded as mobile code to communicate as peers with each other and with an object in the server domain.

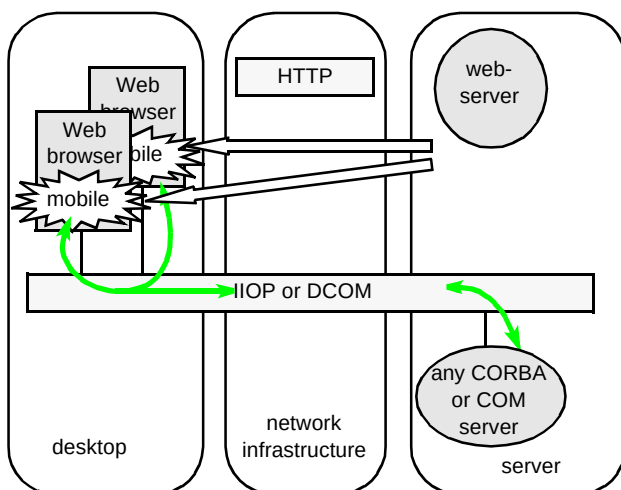


Fig 10 Integration using applet intercommunication.

outlined above (and probably others that have been overlooked), all bootstrapped through the Web infrastructure. Thus in theory we have distributed objects on the Web. However, the remaining obstacle to large-scale flexible deployment is the need for some crucial standards to emerge.

5. Essential standards

In other living creatures ignorance of self is nature; in man it is vice.

Boethius

For large-scale deployment of applications based on a distributed object system in any vertical market, there will always be a need for standardisation of the interfaces presented by the various generic objects that meet those market needs. The Object Management Group is geared up to do this for the OMA, the Active Group (a subset of the Open Group) for ActiveX, and Java APIs will presumably be relinquished to open standardisation by Sun, eventually. In fact, interoperation would be helped by separating the characteristics of standard object interfaces from their object model to the maximum extent possible. These could then be standardised under a single body for each vertical market.

However, standard croutons are of very little use without standard soup. Mackay [14] provides an excellent, minimalist, if slightly dated, agenda for standardisation (whether *de jure* or *de facto*) of the infrastructural soup in which these crouton objects will be immersed. The updated list is as follows:

- Web server API — for server side gateways avoiding the inefficiencies of CGI (no moves towards this),
- object access URL scheme naming,
- client-side state management — the Web is otherwise stateless (this is all but done now that the *de facto* standard 'Cookie proposal' [15] is standards track),
- embedded objects (the document object model for the Web) — this is far more controversial with each camp very entrenched behind large investments; however, there is a proposal [16] to at least sort this out for hypertext markup language (HTML),
- object model gateways — work on COM/CORBA integration by the OMG and Microsoft is proceeding well,
- Java stability — only time can bring this,

- popular object protocols included in the desktop, not essential, but best for performance — of the two most prevalent browsers, Netscape Navigator™ includes IIOP support in its foundation classes from release 4.0, and Microsoft Internet Explorer™ will undoubtedly eventually include DCOM support with the non-Windows ports,
- popular object protocols accepted in network infrastructure — firewalls need to allow them through (if they become popular, this will happen).

Thus it appears that soon it will be easier to put distributed objects on the Web — in practice as well as in theory. It is now time to explore how we might achieve a global web of distributed objects.

6. Some answers

Men's minds perceive second causes,
But only prophets perceive the action of the First Cause.
Jalal-uddin Rumi.

What is the difference between distributed objects on the Web and a web of distributed objects? The difference is that the former involves transplanting object-based applications on to the Web and allows us to start interconnecting these together. The latter is the result of interconnecting all the object systems. The dangerous assumption here is that, once we have an interconnected object system at the scale of the Web, it will still work! This brings us back to the unresolved scale-related issues that distributed object systems appear to face.

6.1 Flexibility versus performance

The answer here lies in solutions that fix the right two (or more) problems in one layer of abstraction.

The Web offers an examples in the URL; the Internet Domain Name Service (DNS) is another. The inspired Naming Authority Pointer (NAPTR) proposal for Uniform Resource Names (URNs) [17] promises to be yet another example of a well-concocted abstraction — URNs unify URLs and DNS very powerfully and elegantly.

However, the object models do not appear to exhibit such inspired performance hacks. The OMA is obsessed with separation of roles within the infrastructure to allow creative competition between ORB vendors. Nonetheless, it is not logical separation that hits performance, only physical separation. Therefore total location transparency is usually the enemy of such performance hacks. The COM has useful 'location translucency' — the DCOM protocol provides a flag so that the programmer can tell whether a call is local or remote. Also, related functions tend to be bundled together in the COM because it is of much less granular

design than the OMA in the first place (as it was more concerned with providing an integrated package for the service of application developers). At the other extreme, native Java has no location transparency at all — the application programmer has to explicitly request remote processing using remote method invocation (RMI), otherwise local is assumed. The most novel examples of flexibility with performance are likely to come from mixing more Web concepts into object models. Java's URL class is a good example. A loader within the virtual machine for distributed classes accessed through URLs would be another.

6.2 Binding complexity

Discovering the best object (or at least one that is good enough) to bind to is probably both the most important and the most difficult problem to be cracked at the complexity of very large scale.

Table 1 gives the results of an exercise to identify the steps required to discover and access services on the Web (see also Fig 11). They are described in generic terms familiar to those who work in the open distributed systems field, then in Web terms alongside. This provides some insights into how closely the Web can be construed to be compliant with the ISO/ITU RM-ODP [1]. The big assumption (cheat) made is that systems designed for human as against machine interaction are comparable. If the comparison holds up (which it does), it implies (as a generalisation) that the Web is merely deficient in the structure of its data, rather than in its functional elements.

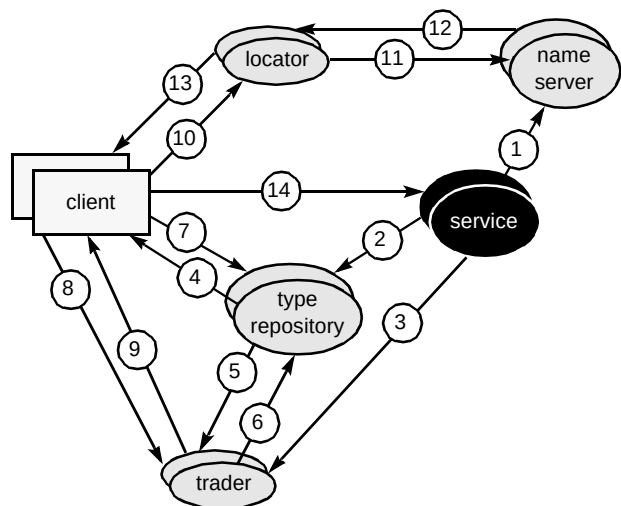


Fig 11 Steps in service discovery and access in ODP Reference Model terms (the fact that all objects may be replicated is represented by each object appearing twice).

The concepts 'service', 'service type', 'service discovery' and 'service access' are used here as defined in Vogel et al [18], and the 'type repository' function and functions of 'trading', 'locating' and 'naming' are defined in the RM-ODP [1].

Table 1 Service offer through to access via discovery.

	Generic distributed system	Web implementation	Insights
	Service export (some parts may have to repeat regularly for dynamic services)		
1	The service must have its name(s) registered against its address(es) in the name server (pre-supposing allocation of these uniquely)	A name for the service is placed somewhere that potential clients already go (e-mail, Web document), with the URL beneath it (i.e. a hyperlink)	A page of hyperlinks is a name-server (albeit unidirectional) and the Web is a massive scale federation of name servers
2	Type(s) need to be chosen by which it is sensible to classify the service and which are known of by the (extensible) type repository	The service is classified by the provider and gradually by those that use it, usually in their natural language, but also in the classification graphs of popular indexes	Natural language descriptions are a major step towards machine-readable classification systems and point to the need for classification in many alternative contexts
3	The service needs its name to be registered against the type of service on offer in the trader	Hyperlinks to the service are gradually placed in popular indexes, in relevant Web pages, news postings and e-mails	The Internet is a massive scale federated trader database from the point of view of services
	Service discovery		
4, 5, 6, 7	The client needs to choose a type of service (and possibly properties) which describes what is required and exists in the type repository (which may interact with the trader for sub-types)	The user browses down an index choosing the most likely categories presented or scans the results of a search engine query (a few tries of the links out of the index may be required before a good path is found)	It is impractical to propose global standards for each service type definition. In a massive scale federation the service types can only have meaning within a context that contains that service
8, 9	The client receives a service name(s) from the trader that best satisfies the requirements description	A resource is found which appears to have the right context and contains a hyperlink with a tempting name	The Web is a massive scale federated trader database from the point of view of clients
10, 11, 12, 13	The client puts the service name(s) to the locator which interrogates the name server to find the address(es) to which these names map; the locator then decides on the best service address for this particular request and returns it to the client.	The hyperlink does not directly access the service required, but presents choices of hyperlinks with the relative merits of each described. The user chooses the 'best' hyperlink (closest, fastest)	Parts of the Web already act as locators (although this is not at all prevalent)
	Service access		
14	The client uses the address to contact the service	The hyperlink chosen directs the client to the service address underlying it	-

The insights in Table 1 show that the challenge is to find a middle path where there is the incentive for content providers to finely spread (and maintain) structured data (or at least metadata — data about data) around the Web that is interconnected with the unstructured data. Put the other way round, 'Every object should have a Web page' [19].

This will allow machines to appear to make sense of the unstructured data. This will be a more productive path while we do not have systems that are intelligent enough to interpret unstructured data.

It would be expected that the Java object model would be the closest to the Web's model and therefore the best candidate for this fine spread of structured objects in the fabric of the unstructured Web data. However, as Java is currently used, it is the furthest from the web of distributed objects model. The problem is that Java objects can only be

deployed in little incestuous clumps that interact with each other locally, unless RMI is used explicitly.

However, because the Java virtual machine can link objects dynamically, its class loader could be overridden to allow class names with global as well as local context (assuming a class name server system). This would explode the bindings between the clusters of classes in Java applets, and make the Java object model have a more appropriate distribution granularity. This would not necessarily suddenly make Java work at large scale, without applying the other suggestions being put forward here.

6.3 Gridlock avoidance

All object models contain either explicit or implicit assumptions about their permitted object interaction models at their core. Interaction models were introduced in the previous section. A more general list follows:

- asynchronous (oneway, ‘fire-and-forget’, maybe, or datagrams),
- idempotent (asynchronous, but where receipt of one message has same effect as many),
- synchronous (request/reply),
- deferred synchronous (request is asynchronous, but connected to a ‘right’ to a reply through storage of some state),
- call-back (retroaction or notification — as with deferred synchronous, but unbounded number of replies possible into the future — used for such applications as publish subscribe, event monitors, etc),
- autonomous agent (a richer form of deferred synchronous).

One sign of a scalable object model is one that allows some degree of decoupling across time. If every request call blocks while waiting for a reply, a totally interconnected world would become very much like the housing market, with long chains of buyers and sellers, all continually switching from a state of hopeful expectancy to a disappointed back-off.

The least tightly coupled model is probably that of the various proposed autonomous agent infrastructures. CORBA has three interaction models — oneway, synchronous and deferred synchronous. The COM, being based on DCE RPCs [7], supports maybe, idempotent and synchronous.

Related to time coupling is the gridlock avoidance problem. This is where a chain of calls end up in a loop with an object unable to answer an incoming request until it has the reply to the request it sent out earlier (which is the cause of the incoming request) (see Fig 12).

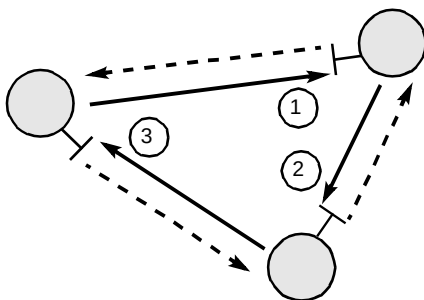


Fig 12 A gridlocked loop of calls.

This is a difficult one to solve. Decoupling timings can help by using deferred synchronous calls, but this does not help if the loop is at the information level (the information in the reply is needed, rather than just the fact that the reply occurred).

DCOM uses ‘causality IDs’ in every message to trace a chain of cause and effect. The Relationship Service is a CORBA service that could be used to map the topology of call patterns, where it is deemed this might be a problem (though this is not what it was originally envisaged for).

It remains to be seen whether such solutions will scale up to global proportions. The DCOM solution requires the object itself to check for offending causality IDs, which could put a strain on performance. The suggested CORBA solution would require a network of co-operating relationship servers at large scale which would have to be kept up to date with every interaction deemed sensitive to gridlock, although it is possible that the topology map need not be completely up to date in order to head off or at least help prise open a deadlock.

6.4 Build complexity

So far, management of complexity at run time has been considered. At build time, developers also need help with the sheer volume of APIs they have to understand in depth in order to write good code (and the problems they cause when they do not understand). The ideal solutions here would enable applications to just consist of the functional bits. Separately, specialists in each non-functional field could write ‘metaobjects’ which provided non-functional services. Yet someone else could integrate the two without the need to alter the source code of either.

Microsoft have a reputation for good support of the development process. This is most fundamentally provided by the pervasive assumption in the COM that, across the industry, different parties are developing components for applications without any co-ordination. Hence, support is built in for checking which version of a component has been accessed, and whether it can do what the expected version could do. Further, Microsoft’s Transaction Server product [20] allows separation of the build of the functional part of an application from the build of the non-functional transactional mechanisms and their policy configuration. The techniques being researched for Reflective Java [21] and MetaJava [22] extend this capability to other non-functional requirements and they allow dynamic re-configuration for which Microsoft’s product is not designed.

6.5 Non-deterministic reliability

The generally fuzzy nature of the answers so far revealed from the Web’s success at scaling hints at probably the least obvious lesson that object systems designers need to learn — the Web scales well because it does not work properly.

Another lesson the Web teaches us is that a ‘bomb-proof Web’ can be created from unreliable component parts [23].

It may appear heresy, but if BT offers a service over the Web and it fails, the customer has the opportunity to find another similar service elsewhere in the world. Thus from the customer's perspective, the service reliability of the product they originally acquired from BT is improved simply because it exists within the Web infrastructure!

Thus one of the implicit problems with most object models is that they are too deterministic. For instance, one could envisage designing the Web in such a way that every reference to a resource (e.g. by hyperlink from other resources) was recorded so that if a resource moved all references could be updated. The fact that many of the references are held by computers that are powered off more than powered on, would put enormous strains on the infrastructure to hold all these pending reference updates. This is not to say that such a project would not be worthwhile (e.g. HyperG [24]); however, the loose coupling of the Web was what made its federated growth possible. The W3Objects [25] approach preserves this loose coupling, while the NAPTR URN proposal [17] neatly solves the problem by adding an extra, scalable layer of abstraction — naming.

The reference counting dilemma is a sensitive area in many distributed object models. Because distributed COM has recently evolved from the non-distributed COM, it has kept very tight reference counting, not only to allow object relocation, but also to allow objects to free up resources when there are no more clients using them. Thus if a reference to an object is given to one client which then passes it on to yet another client, the infrastructure keeps count of both these client references. Because partial failure is an occupational hazard of distributed systems, the default DCOM behaviour is for the client to be 'pinged' regularly to check the reference is still 'booked out' to a living client. Although there are clever schemes for aggregating pings and sending only 'deltas' to ping messages, this traffic would obviously bring down a large-scale system. It is possible to specify that this behaviour is turned off, leaving it to the application programmer to devise distributed failure strategies (or not).

The general approach taken by the OMA is not to reference count at all. Instead clients must fall back to address-mapping services if they discover an object has relocated. The issue of how an object knows whether it is safe to delete itself and free up resources is left as an exercise for the programmer. In other words, neither major object model has a real solution to reference counting and both are likely to 'fail' in large-scale systems, but for opposite reasons — DCOM would become congested while the OMA uses more resources than strictly necessary. It was a conscious decision by the OMG to leave garbage collection as application specific — the only one that made sense at

scale. It is interesting that an architecture specifically designed for large-scale distribution suffers from a similar inherent problem as the Web.

7. Conclusions

Truly, 'Only he that rids himself forever of desire can see the Secret Essences.' He that has never rid himself of desire can only see the Outcomes.

Lao Tzu

All the technologies and standards are falling into place that will allow islands of distributed object systems to be deployed over the ubiquitous Web infrastructure — distributed objects on the Web.

The tendency to improve the capabilities of any one system by connecting it to others will inevitably lead to the interconnectedness of every object system. However, there is no guarantee that a global object system will work.

It seems probable that strictly deterministic systems will remain as islands of a certain bounded size, at least into the medium term.

If the lessons that the Web can teach us are correct, a global distributed object system might have to be redesigned to be deliberately less reliable at the micro level in order to function reliably at the macro level.

The alternative is to spend a lot more time analysing and radically redesigning the existing distributed object systems to eliminate the possibility of some phenomena like large-scale gridlock or congestion.

The former approach is likely to deliver the advantage of an interoperating global object system sooner than the deterministic alternative.

The middle way of highly reliable islands interconnected by 'fusible links' is probably a workable compromise.

The unstructured data on the Web is generally unsuitable for machine processing. A solution to this problem would massively increase the utility of an already highly popular resource. Rather than focusing on the development of techniques for machines to understand natural language, the goal of machine navigation could be achieved by introducing incentives to content providers to finely spread (and maintain) more structured metadata objects within the fabric of the existing Web, i.e. a web of distributed objects.

Acknowledgements

Acknowledgements are due to Ben Crawford, BT, for reviewing this paper so thoroughly, and Andrew Watson for his very useful draft paper 'Comparison of Object Models', ANSA ref APM.1530.00.01 (8 Sep 1995).

References

- 1 'Open Distributed Processing Reference Model (RM-ODP)', ISO/IEC 10746-1 to 10746-4, or ITU-T X.901 to X.904 (January 1995), <http://www.iso.ch/isob/switch-engine-cate.pl?searchtype=refnumber&KEYWORDS=10746>
- 2 'Object Management Architecture Guide', Object Management Group, Inc.
- 3 'CORBA Services; Common Object Services Specification', Object Management Group, Inc (Revised Edition March 1995) (updated November 1996), <http://www.omg.org/library/corbserv.htm#dprin>
- 4 'CORBA 2.0, Universal Network Objects', PTC/96-08-04, Object Management Group, Inc (July 1996), <http://www.omg.org/corba/corbiiop.htm>
- 5 'The Component Object Model Specification', Draft Version 0.9, Microsoft Corporation and Digital Equipment Corporation (October 1995), <http://www.microsoft.com/oledev/olecom/title.htm>
- 6 Brown N and Kindel C: 'Distributed Component Object Model Protocol — DCOM/1.0', Microsoft Corporation (November 1996), <http://ds1.internic.net/internet-drafts/draft-brown-dcom-v1-spec-01.txt>
- 7 CAE Specification: 'X/open DCE: Remote Procedure Call', X/Open Company Limited, X/Open Document Number C309 (1994), http://www.osf.org/mall/dce/free_dce/htm (after registration)
- 8 Merle P: 'Integrating CORBA Objects in the WWW', Université de Lille (May 1996), http://www.lifl.fr/~merle/papers/96_WWW5/panel/slides.html
- 9 McCool R: 'The Common Gateway Interface', (1993), <http://hoohoo.ncsa.uiuc.edu/cgi/>
- 10 Rees O et al: 'A Web of Distributed Objects', APM Ltd (July 1995), <http://www.ansa.co.uk/ANSA/ISF/wdist/obj/Overview.html>
- 11 Berners-Lee T, Masinter L and McCahill M: 'Uniform Resource Locators (URL)', RFC 1738, CERN, Xerox PARC, University of Minnesota (December 1994), <http://ds.internic.net/rfc/rfc1738.txt>
- 12 Fielding R et al: 'Hypertext Transfer Protocol — HTTP/1.1', RFC2068 (August 1996), <http://ds.internic.net/rfc/rfc2068.txt>
- 13 Kindel C: 'The "clsid:" URL Scheme', Microsoft Corporation, Draft (28 February 1996), <http://www.w3.org/pub/www/Addressing/clsid-scheme>
- 14 Mackay J: 'The Web meets CORBA the Geek', Andersen Consulting (March 1996), http://www.w3.org/pub/WWW/OOP/9606_Workshop/submissions/37.2.html
- 15 Kristol D M, Montulli L and Netscape Communications: 'HTTP State Management Mechanism', Bell Laboratories, Lucent Technologies and Netscape Communications, Draft 5 (November 1996), <ftp://ftp.ietf.org/internet-drafts/draftietf-http-state-mgmt-05.txt>
- 16 Raggett D et al: 'Inserting objects into HTML', W3C, Microsoft Corp, Netscape Communications Corp, Spyglass Inc, Sun Microsystems, and Pathfinder (Draft) (April 1996), <http://www.w3.org/pub/WWW/TR/WD-object.html>
- 17 Daniel R and Mealling M: 'Resolution of Uniform Resource Identifiers using the Domain Name System', Los Alamos National Laboratory and Network Solutions, Inc (November 1996), <ftp://ftp.ietf.org/internet-drafts/draft-ietf-urn-naptr-01.txt>
- 18 Vogel A, Beitz A and Iannella R: 'Discovery and Access of Services in Globally Distributed Systems', DSTC Pty Ltd (May 1995), <http://www.dstc.edu.au/AU/staff/andrea/s-vogel/papers/ulpaa95/ulpaa.html>
- 19 Herbert A, APM Ltd, private seminar (1996).
- 20 Microsoft Corporation: 'Microsoft Transaction Server White Paper', (January 1997), <http://www.microsoft.com/transaction/>
- 21 Zhixue W and Schwiderski S: 'Reflective Java: Making Java even More Flexible', APM 1936 (February 1997), <ftp://ftp.ansa.co.uk/phase3-doc-root/ex/APM.1936.ps.gz>
- 22 Kleinöder J and Golm M: 'An Efficient Run-Time Meta Architecture for Java: MetaJava', Proceedings of the International Workshop on Object Orientation in Operating Systems — IWOOS'96, Seattle, Washington (IEEE) (October 1996), <http://www4.informatik.uni-erlangen.de/Projects/PM/Java/>
- 23 Cardelli L: 'What is the Web's Model of Computation?' Digital Equipment Corporation, Systems Research Centre (June 1996), http://www.w3.org/pub/WWW/OOP/9606_Workshop/submissions/32.html
- 24 Andrews K, Kappe F and Maurer H: 'Serving Information to the Web with Hyper-G', WWW'95, Darmstadt, Germany, Computer Networks and ISDN Systems, 27, No 6, Elsevier Science, pp 919—926 (1995), <http://www.igd.fhg.de/www/www95/papers/105/hgw3.html>
- 25 Ingham D, Caughey S and Little M: 'Fixing the 'Broken-Link' Problem: the W3 Objects Approach', University of Newcastle upon Tyne (May 1996), http://www5conf.inria.fr/fich_html/papers/P32/Overview.html



Bob Briscoe joined BT in 1980 as a sponsored student, attending the University of Cambridge from 1981 to 1984, where he gained a BA in Engineering and an automatic MA in 1986. On taking a post in the Product Design Unit at BT Laboratories (BTL), he was involved in various mechanical and electronic hardware design projects until 1991. Overlapping this from 1987, he became system manager of the unit's Computer Aided Design and Office Automation facilities. In 1991 he became system manager for all the Computer Aided Engineering facilities of the sites around BTL which developed into a multi-platform and multi-protocol intranet run as a truly integrated distributed system under progressively devolved access rights. Late in 1994 he joined the Distributed Systems Group in BT's Advanced Research department where he specialised in future information services architecture and infrastructure, particularly application layer protocols and internationalisation of Internet electronic commerce. He is BT's technical representative for the ANSA project, an international distributed systems research consortium, and a technical advisor to the Internet Law and Policy Forum, an international research and lobbying body.

